

Chapter 3

Methodology and Requirements Analysis

3 Methodology and Requirements Analysis

This chapter describes the methodology and requirements analysis for the proposed system. It explains the development approach used in the project and identifies the stakeholders involved in the system. The chapter also presents the functional and non-functional requirements needed to support the system features. In addition, system models such as the use case diagram are introduced to illustrate how users interact with the system and how the main system functions are organized.

3.1 Methodology

3.1.1 Methodology Used: Agile

The project adopts the Agile methodology, which supports iterative development, continuous evaluation and incremental improvement. Instead of developing the whole application in a single stage, Ethernacare was divided into smaller functional areas, including authentication, Oren care, rolling check-ins, emergency escalation, trusted contacts, virtual rewards, Legacy Planning, AI guidance and weather integration. Each area could be designed, implemented and tested separately before being integrated with the rest of the application.

3.1.2 Methodology Phases

1. **Requirement Gathering** – The first phase identified the functional and non-functional requirements of Ethernacare, including configurable inactivity thresholds, Oren interaction, staged SMS escalation, verified contacts, virtual rewards, protected Legacy access, authentication, biometric session unlock and AI guidance.
2. **System Design** – The application architecture, database model, user interface, activity flows and security controls were designed. The design also identified the responsibilities of Flutter, Supabase, Android WorkManager, Edge Functions, scheduled processing and external providers.
3. **Development** – The application was implemented module by module using Flutter and Dart. Supabase was used for authentication, PostgreSQL data storage, Row Level Security, private file storage, Realtime updates, Edge Functions and scheduled server processing.
4. **Testing** – Automated unit, service, repository and widget tests were conducted to verify deterministic business rules and user-interface behaviour. Static analysis and platform builds were also performed. Functions involving real SMS receipt, GPS accuracy, biometric hardware and exact Android scheduling require separate physical acceptance testing.

5. **Evaluation and Improvement** – Test results and application behaviour were reviewed to identify problems. Requirements, validation rules, interfaces and server functions were refined through repeated Agile iterations until the main application features operated consistently.

3.1.3 Justification for Methodology

The Agile methodology is suitable for this project because the system consists of multiple independent modules, including activity monitoring, emergency alerts, AI chatbot guidance, and document storage. These modules can be developed and tested separately in iterative cycles. Additionally, system requirements may evolve after usability testing with potential users, especially elderly individuals, making a flexible development approach necessary. Iterative development helps reduce project risks by identifying issues early and allowing continuous improvements. It also enables critical features, such as emergency alert functions, to be tested and refined at an early stage of development.

3.2 Requirements Analysis

3.2.1 Stakeholders

Primary User

The primary user is an older adult or independent individual who lives alone. The user registers or signs in, completes profile and consent setup, verifies a phone number, configures an inactivity threshold, checks in through Oren, manages trusted contacts, uses rewards and AI guidance, and records Legacy Planning information.

Primary Trusted Contact

A trusted contact is an emergency and Legacy recipient, not a second authenticated app role. One verified contact is selected as primary. That person may receive escalation messages and, after separate SMS verification, may view funeral preferences at any time and protected Legacy Notes or documents only during an authorised release window.

Reward Administrator

An authorised administrator uses a separate admin route to create, edit, refresh and delete eligible virtual reward catalogue entries. The administrator does not gain access to personal user, contact or Legacy information.

System Administrator / Developer

The system administrator maintains Flutter releases, Supabase migrations and policies, Storage, Edge Functions, Cron jobs, provider secrets, logs and backups. Production credentials are stored as server secrets and are not embedded in the mobile application or report.

3.2.2 Functional Requirements

Module 1: Authentication, Onboarding and Profile

Users register with email and password or a configured Google OAuth provider. Email sign-up and password recovery use Supabase confirmation flows. New users accept the Terms and Conditions, complete mandatory profile fields, verify the user phone by a six-digit SMS OTP and add a verified primary contact. After setup, a skippable eight-step guide introduces the main modules; it appears once per account and can be replayed from Profile or Settings. OAuth profile bootstrap ensures that the public user row uses the same authenticated UUID.

Module 2: Oren Check-In and Care

Oren provides the explicit heartbeat action. A successful check-in is allowed when the configured rolling threshold has elapsed, resets the current inactivity cycle and updates history and rewards. Oren energy decays by one point for every complete hour, feeding restores energy, playing consumes energy, and owned toys and token state persist per user on the same device. Care and shop mutations are serialized so rapid or simultaneous taps cannot overwrite energy, token, ownership or selected-item changes.

Module 3: Inactivity and Emergency Response

The inactivity threshold can be set from one to 168 hours. At one missed window the status becomes overdue and a local reminder is created. At two missed windows the cloud worker attempts an SMS to the verified user phone. At three missed windows it creates one inactivity alert and attempts one SMS to the verified primary contact. The server worker runs every 15 minutes and uses persisted state and idempotency to prevent duplicate escalation. SMS provider requests have a 12-second timeout, interrupted processing can be recovered after five minutes, and the emergency SMS endpoint accepts only verified recipients and approved Ethernacare message categories. Manual SOS can record location and send a contact alert; calling 999 always requires explicit user action.

Module 4: Trusted Contacts and Location

Users can manage up to five owned contacts and choose exactly one primary contact. Phone numbers must be verified by SMS OTP before they can be saved. Emergency records can include a Maps link when location permission and a valid position are available.

Module 5: Virtual Rewards and Oren Shop

Check-in milestones unlock virtual badges and vouchers. A badge is stored in My Badge List when first collected, while a voucher displays a stable user-specific redemption code. An authorised admin maintains the catalogue through a separate route. Oren tokens are a separate local currency used to buy toys and care items.

Module 6: Weather and AI Guidance

Weather first follows the Malaysian state and region saved in the profile through the data.gov.my weather service, with device coordinates and Open-Meteo as fallback. AI Guidance uses a secured Edge

Function and provider fallback for general information only; it does not diagnose conditions or replace professional medical, legal or emergency advice.

Module 7: Legacy Planning and Checking

The owner records funeral preferences, protected Legacy Notes and private PDF/JPEG/PNG documents up to 10 MB. Notes reject common password, PIN, OTP and recovery-secret patterns. Funeral preferences are available to the verified primary contact after a valid Legacy SMS OTP. Notes and documents require 90 days without check-in, a 24-hour owner warning/cancellation period and an active seven-day release window. Secure documents use short-lived signed links.

Module 8: Biometric Session Lock

A signed-in user can enable biometric protection on a supported device. The app requests local authentication when restoring or resuming that user session. Asynchronous prompt coalescing prevents overlapping biometric requests, and a cancelled or failed prompt keeps the application locked.

3.2.3 Functions not included

Funeral Service Booking and Payment System

While the application does not include a marketplace or transaction system. Users cannot browse specific funeral packages, service providers, or make payments for funeral services directly through the app.

Direct Communication with Service Providers

The application does not include a real-time chat or messaging system to connect users with funeral directors, lawyers, or religious organizations. The AI chatbot provides general information only and does not facilitate external business communication.

Medical and Biological Health Monitoring

The system relies solely on phone activity (screen interaction and app usage). It does not integrate with wearable devices (like smartwatches) to monitor heart rate, falls, or other biological vital signs. It is not a medical diagnostic tool.

Legal Will Drafting and Notarization

The application does not provide the legal tools to create, draft, or notarize a will. Users must obtain a legally binding will through traditional legal channels before uploading a copy to the app.

Direct Integration with Emergency Dispatch (999/911)

The system is designed to alert trusted contacts and family members first. It does not automatically dial or trigger a direct dispatch from government emergency services (Police or Ambulance) to prevent false alarms that could lead to legal liabilities or resource wastage. Except the user triggers the emergency call manually else the emergency services won't trigger.

3.2.4 Non-Functional Requirements

Reliability

Server-side inactivity evaluation shall run every 15 minutes and persist each reminder stage. SMS work shall be queued with idempotency and retry state so repeated worker runs do not create duplicate escalation. Android local monitoring remains best-effort and shall not be described as exact-time or continuously running.

Security and Privacy

Supabase Auth shall identify users; Row Level Security shall restrict owner data; private Storage shall protect documents; Edge Functions shall keep provider secrets off the client; phone and Legacy access shall use expiring OTPs with attempt limits; and biometric authentication shall protect only the local restored session rather than replace server authentication.

Usability and Accessibility

The interface shall provide readable text, clear status/action differentiation, scrollable forms, guidance controls, safe error messages and responsive layouts. Device settings include text-size choices and reduced-motion support, while important controls expose useful semantic labels. Threshold, escalation, contact verification and Legacy release states shall be explained in plain language. These choices follow Flutter guidance that release checks should cover screen readers, tappable targets, contrast and usability at large text/display scale factors (Flutter, 2026).

Performance and Efficiency

Weather, reward, chat and dashboard data shall use bounded caching where appropriate. Realtime subscriptions and manual refresh shall update shared catalogues without forcing unnecessary full-page reloads. Background work shall minimise battery usage and network calls.

Maintainability and Portability

The Flutter client shall separate presentation, services, repositories and models. SQL migrations and Edge Functions shall be versioned and idempotent. Core deterministic rules shall be covered by

automated tests, while Android hardware and carrier behaviours shall have documented physical acceptance cases.

3.3 Use Case Diagram

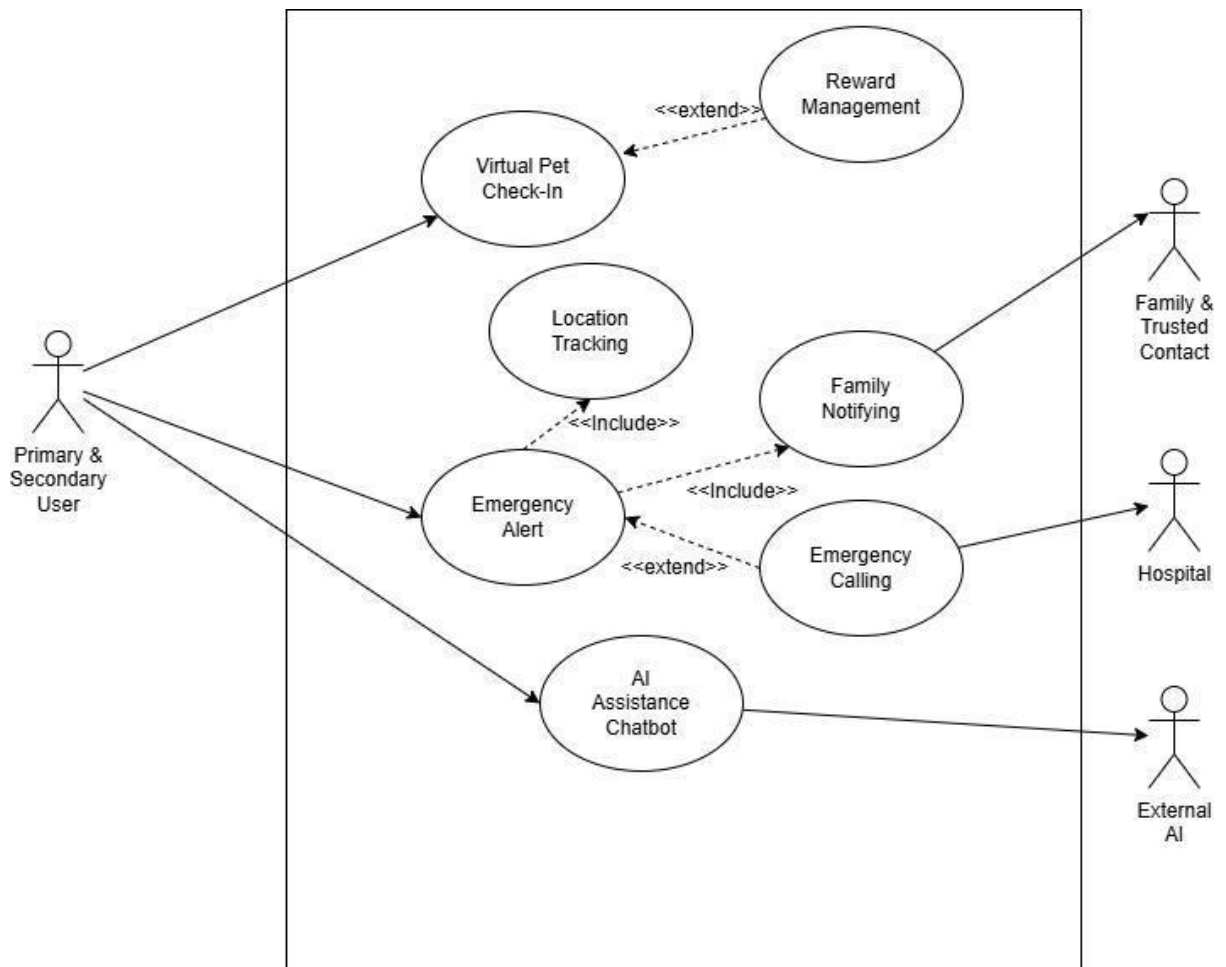


Figure 3.1: Use Case Diagram for each of stakeholders

3.3.1 Use Case Actors

Use Case Actors

Primary User - authenticates, completes onboarding, manages the profile and contacts, performs Oren check-ins and care, views weather and rewards, asks AI Guidance, manages Legacy Planning, and triggers manual SOS.

Primary Trusted Contact - receives eligible SMS/email notifications and uses the public Legacy Check flow with the correct Legacy UID, verified primary phone and separate OTP. This actor does not log in to the owner account.

Reward Administrator - signs in through the separate admin route and maintains virtual reward catalogue entries within admin database policies.

External Services - Supabase provides authentication, data, private storage and scheduled functions; Twilio handles SMS; Brevo/SMTP handles transactional email; data.gov.my and Open-Meteo provide weather; an AI provider supplies general guidance; and device APIs provide location, local notifications and biometrics.

Main Use Cases

Threshold Check-In - determine whether the rolling threshold has elapsed, insert one check-in atomically when due, reset inactivity state, and refresh rewards and Oren state.

Inactivity Escalation - evaluate elapsed windows every 15 minutes, update the current reminder stage, notify the user at stage two, notify the primary contact once at stage three, and suppress duplicates.

Manual Emergency Alert - ask for confirmation, capture location when available, store the event and attempt the selected contact action. Opening the 999 dialler remains manual.

Legacy Release - send the owner a 90-day warning, allow 24 hours to cancel or check in, open a seven-day protected-content window if not cancelled, and revoke access on expiry or a new check-in.

Virtual Reward Administration - create, edit, refresh or delete an eligible catalogue reward without exposing the administration route in the normal user navigation.

3.4 Chapter Summary and Evaluation

This chapter described the Agile methodology, current stakeholders, implemented modules and system constraints. The requirements now distinguish the rolling threshold from a fixed daily timer, the cloud worker from best-effort Android execution, and notification support from guaranteed emergency response. They also define the current biometric session lock, virtual reward administration and staged Legacy release. These requirements provide a traceable basis for the design and context-based testing in later chapters.

Chapter 4

System Design

4 System Design

This chapter presents the system design of the Ethernacare application. It illustrates how the system is structured and how its components interact to fulfill the required functionalities. The chapter includes the architecture design, class diagram, entity relationship diagram (ERD), and activity diagrams to provide a clear representation of the system's structure and behavior. These design models serve as a blueprint for the system implementation and ensure that the application is well-organized, scalable, and aligned with the defined requirements.

4.1 Architecture Diagram

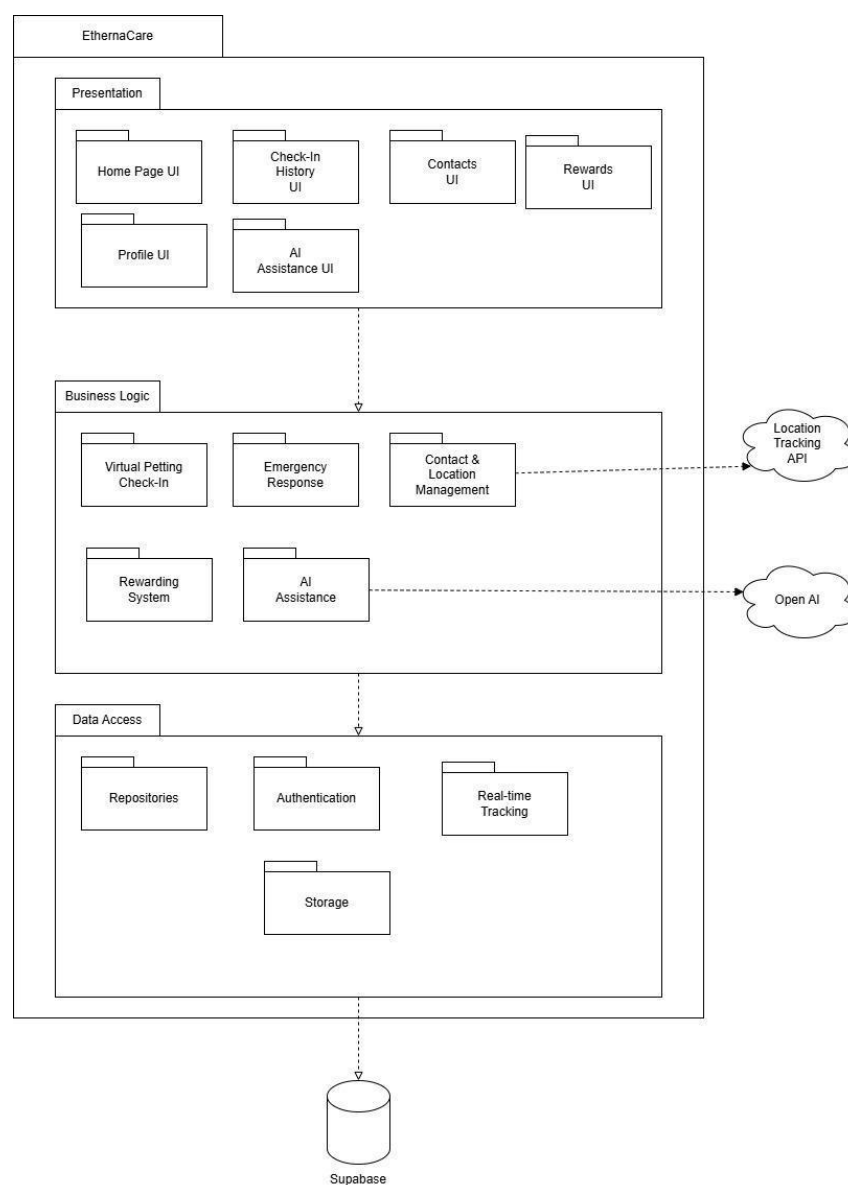


Figure 4.1: 3 Layer + 2 Tier Architecture diagram

4.1.1 Architecture Justification

EternaCare uses a layered Flutter client connected to a managed Supabase backend. The Presentation layer contains responsive screens and reusable widgets. Controllers and services coordinate validation, threshold check-ins, inactivity state, rewards, weather, AI, biometric gating, notifications, SMS and local caching. Repositories isolate Supabase Auth, PostgreSQL, Storage, RPC and Edge Function access. Scheduled Edge Functions provide cloud execution independently of whether the phone is open.

Presentation Layer

The Presentation layer is responsible for handling all user interactions through interfaces such as the Home Page UI, Check-In History UI, Contacts UI, Rewards UI, Profile UI, and AI Assistance UI. This layer is designed to provide a simple, clear, and user-friendly experience, which is especially important for elderly users who may not be familiar with complex applications. By separating the user interface from the system logic, changes to the UI can be made without affecting the core functionality of the system. This improves usability, maintainability, and allows for easier future enhancements such as redesigning the interface or adding new features.

Business Logic Layer

The Business Logic layer applies reusable rules such as rolling check-in eligibility, three-stage inactivity escalation, one-primary-contact enforcement, OTP verification, reward claiming, Oren energy decay, weather fallback, biometric prompt coordination and Legacy release timing. Controllers delegate these rules to services so that they can be tested without the visual interface.

Data Access Layer

The Data Access layer contains repositories for authentication, profiles, check-ins, contacts, emergencies, rewards, Legacy data and private documents. Supabase Row Level Security and storage policies enforce ownership. Edge Functions use server credentials only for narrow privileged operations such as SMS, AI requests and scheduled inactivity processing.

4.1.2 Architecture Advantages

- **Scalability:** Individual layers in the architecture can be scaled independently to meet performance needs.
- **Flexibility:** Different technologies can be used within each layer without affecting others.

- **Maintainability:** Changes in one layer do not necessarily impact other layers, thus simplifying the maintenance. (Geeksforgeeks, 2025)

4.2 Activity Diagram

4.2.1 Login Process

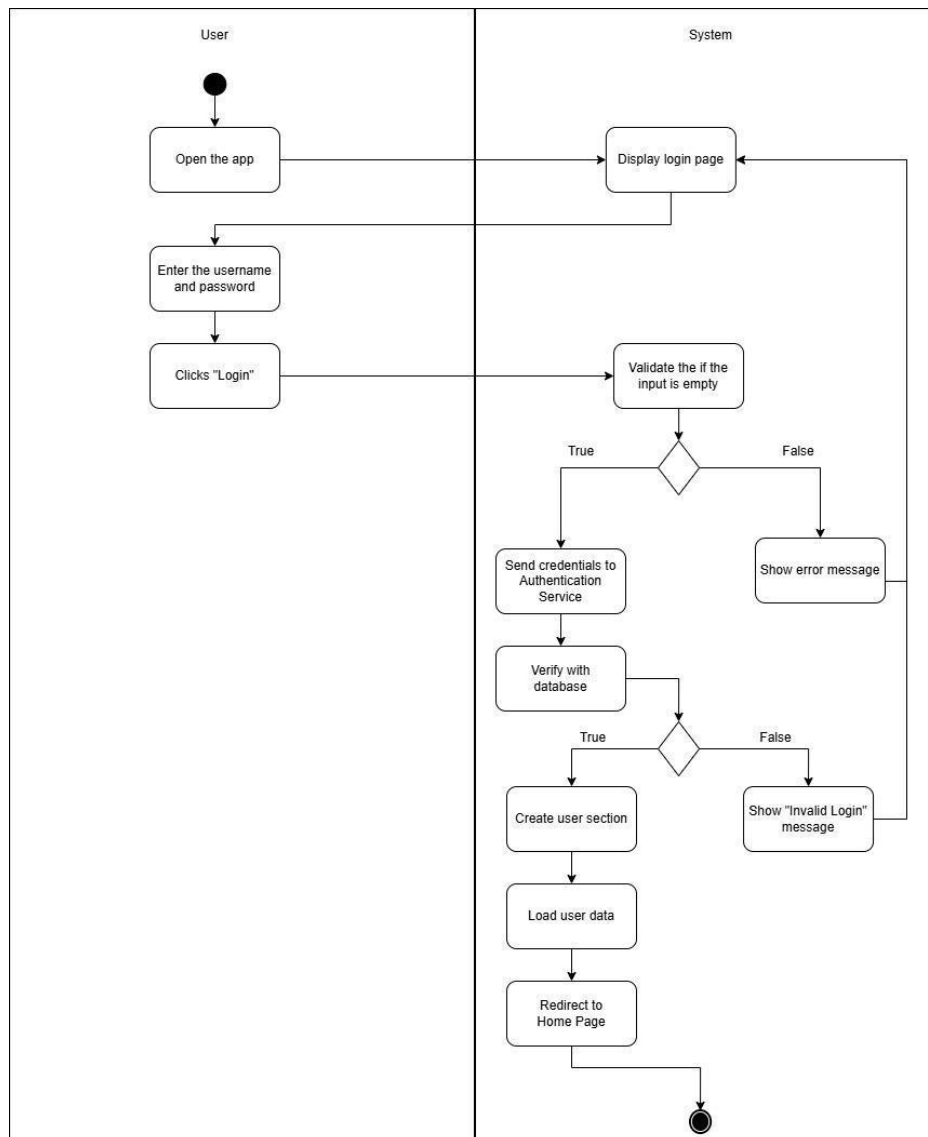


Figure 4.2.1: Activity Diagram show the login process of the Ethernacare

The login flow validates email/password credentials or starts Google OAuth. Supabase creates the authenticated session and the app ensures that the matching public profile exists. First-time accounts continue through Terms acceptance, profile setup, phone verification and primary-contact setup. Returning sessions can be protected by a per-user biometric lock on supported devices before the main interface is shown.

4.2.2 Virtual Petting Check-In

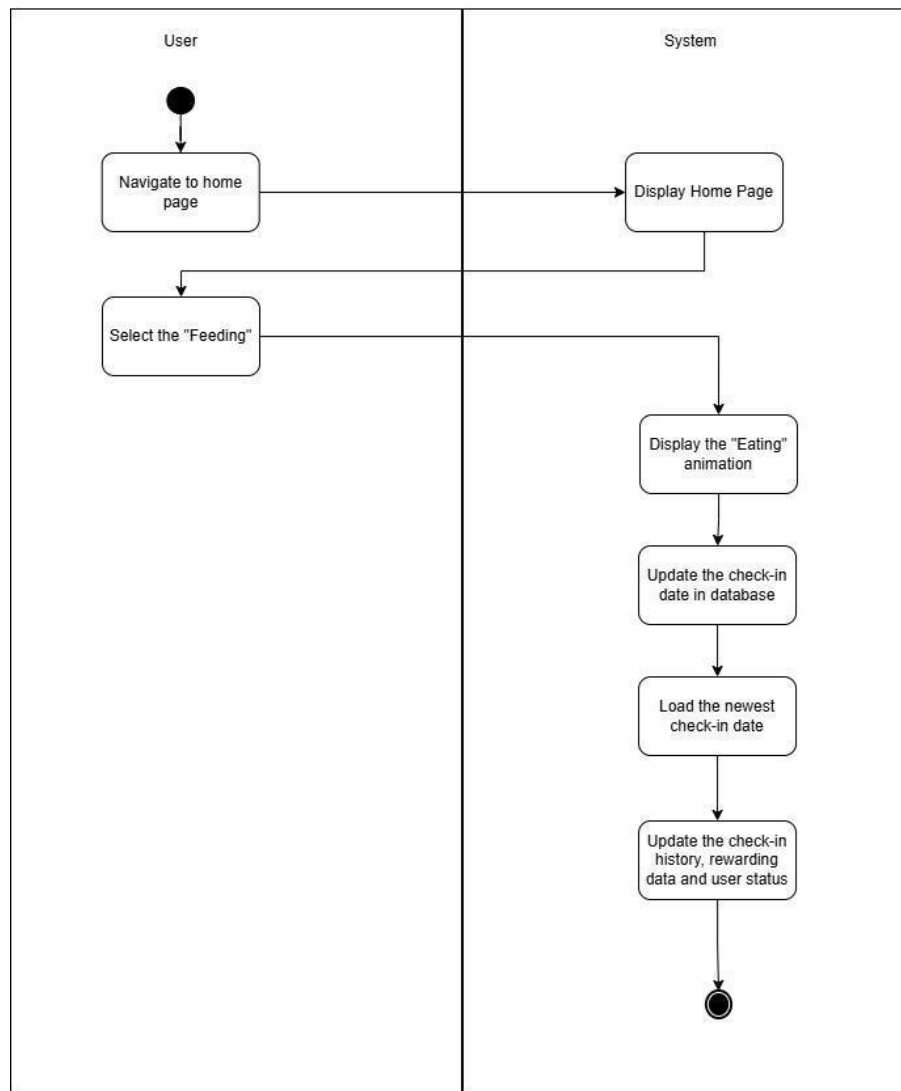


Figure 4.2.2: Activity Diagram show the process of Virtual Petting

The user taps Oren to request a check-in. The repository calculates eligibility from the latest check-in and the configured threshold, then creates one record atomically only when due. Success resets inactivity state, refreshes history and virtual rewards, and applies Oren energy, token, mood, sound and animation changes. Feeding and playing remain care actions and do not create false heartbeat records.

4.2.3 Emergency Alert

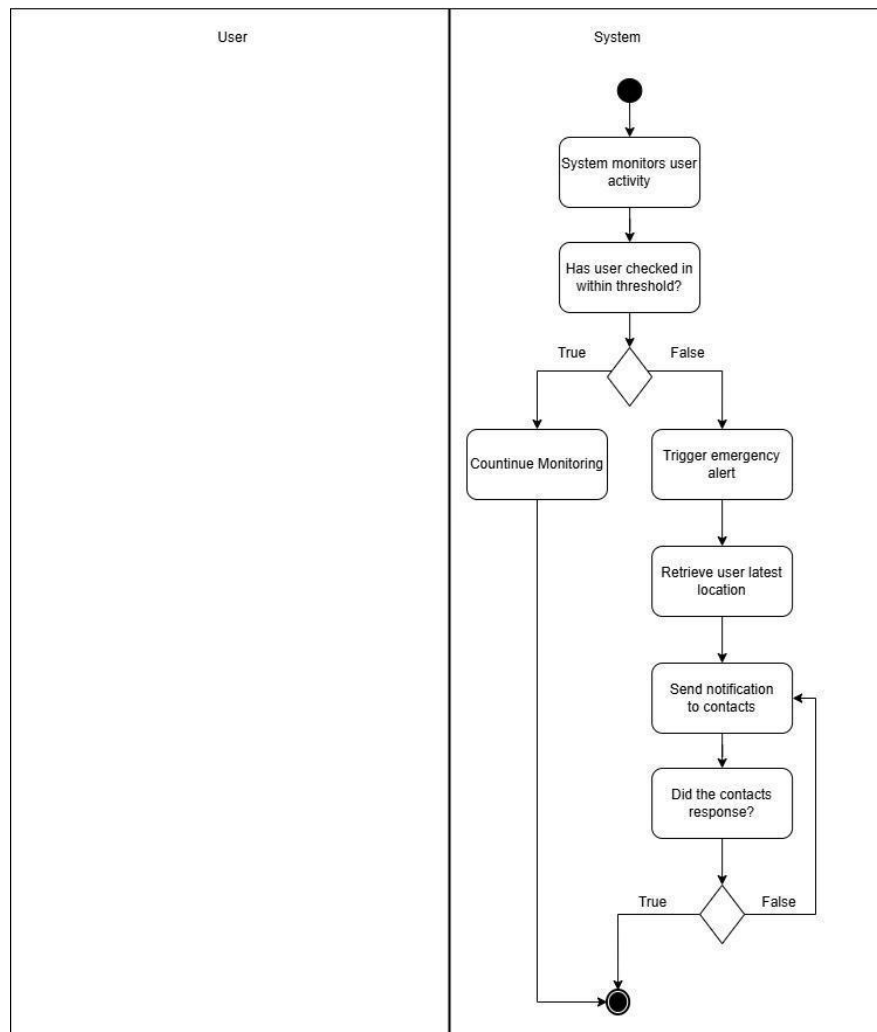


Figure 4.2.3: Activity Diagram show the process of Emergency Alert

The emergency design contains two paths. Manual SOS requires an explicit user action and may attach a location or Maps link when permission and positioning are available. Automatic inactivity processing is evaluated by the server every 15 minutes: the first missed window marks the user overdue and creates a local reminder, the second attempts a user SMS, and the third records one alert and attempts one trusted-contact SMS. Repeated runs are idempotent, and Ethernacare never automatically calls 999.

4.3 ERD Relationship Diagram